

SYSTEMS ENGINEERING CAPSTONE PROJECTS PORTFOLIO

MNC-Style Architectural Frameworks & Granular Task
Breakdowns

Scope: Advanced C Systems, FreeRTOS Embedded & Python HIL Frameworks

Author: Senior Systems Architect & Technical Director

Workspace Root: /home/siva/sivaramireddy/Project1/

Minux Portfolio: Enterprise-Grade Systems Engineering Capstone Projects

This portfolio defines **three high-impact, professional-grade capstone projects** spanning C Systems, Embedded Systems, and Python Automation. Each project is designed from the perspective of an Engineering Manager and Senior Architect to replicate world-class MNC engineering pipelines (like Tesla, NVIDIA, or Qualcomm).



Project 1 (C Systems): MinuxDB - Concurrent Key-Value Database Engine

An elite, high-performance, disk-persisted concurrent Key-Value Database Engine written in pure C11, bypassing standard high-level SQL database libraries.



System Architecture

```
graph TD
    Client["Client Thread (REPL/API)"] -->|Submit Command| Parser["Command Parser & Validator"]
    Parser -->|Aligned Allocation| Arena["Memory Arena / Pool Manager"]
    Parser -->|Write-Ahead Log| Disk["Unbuffered I/O File Descriptor (.log)"]
    Parser -->|Mutex Protected Write| Index["In-Memory Index Tree (Hash/B-Tree)"]
    Index -->|Concurrency Control| RWLock["POSIX Read-Write Locks"]
```



Core Tech Stack & Requirements

- **Language/Standard:** Strict ISO C11 (`-std=c11 -Wall -Wextra -Werror -pedantic`).
 - **Memory Management:** Custom 8-byte aligned Memory Arena (for transient parse tokens) and Object Pool (for database index nodes). Standard `malloc` strictly banned in execution loops.
 - **Systems I/O:** Direct, unbuffered file descriptors (`open` , `read` , `write` , `close` , `fsync`) for Write-Ahead Logging (WAL).
 - **Concurrency:** POSIX Threads (`pthread`), Read-Write Locks (`pthread_rwlock_t`) for thread-safe concurrent index lookups.
 - **Safety:** 100% MISRA C compliance subset, validated via `cppcheck` . Zero memory leaks under Valgrind.
-

MNC-Style Sprint Task Breakdown (Jira Checklist)

Sprint 1: Memory Architecture & Basic Storage

- **[JIRA-CDB-101] Custom Aligned Arena Allocator Core**

- Description: Pre-allocate a 10MB memory pool. Implement O(1) linear allocation adjusting offsets to 8-byte boundaries.
- Deliverable: `arena.h` / `arena.c` containing `arena_alloc` with compile-time assertions.

- **[JIRA-CDB-102] Object Pool Node Allocator**

- Description: Implement a block allocator using free-list bitmaps to serve database entry nodes in constant O(1) time.
- Deliverable: `pool.h` / `pool.c` with 0 external fragmentation.

- **[JIRA-CDB-103] Unbuffered Write-Ahead Log (WAL) Engine**

- Description: Write a persistent logger that records database insert operations directly to disk via raw system descriptors, forcing disk sync (`fsync`).
- Deliverable: `wal.c` managing crash-resilient append-only logs.

Sprint 2: Database Indexing & Command Parsing

- **[JIRA-CDB-201] High-Speed In-Memory Hash Index**

- Description: Build an in-memory hash table using separate chaining for collision resolution, storing key-to-offset maps.
- Deliverable: `index_hash.c` supporting O(1) key lookups.

- **[JIRA-CDB-202] Branch-Free Command Tokenizer**

- Description: Write a state-machine based command string parser that extracts commands (`GET` , `PUT` , `DEL`) with cyclomatic complexity ≤ 3 .
- Deliverable: `parser.c` parsing string queries into structured execution models.

Sprint 3: Concurrency & Lock Disciplines

- **[JIRA-CDB-301] Dynamic Read-Write Locks Integration**

- Description: Protect the Hash Index using POSIX RW Locks, enabling concurrent read lookups but mutually exclusive write modifications.
- Deliverable: Thread-safe hash table execution paths verified under Helgrind.

- **[JIRA-CDB-302] Active Transactions Crash Recovery Manager**

- Description: Implement a recovery boot routine that scans the WAL file, parses uncommitted transactions, and rebuilds the hash index automatically.
- Deliverable: `recovery.c` verifying index reconstruction on dirty boots.

Sprint 4: Performance Audits & Verification

• [JIRA-CDB-401] Cppcheck & MISRA C Compliance Audit

- Description: Run static analysis check over all database modules using MISRA guidelines checkers. Eliminate all warnings.

- Deliverable: Cppcheck terminal report with 0 style, memory, or warning errors.

• [JIRA-CDB-402] Valgrind Leak-Check & Gprof Profiling

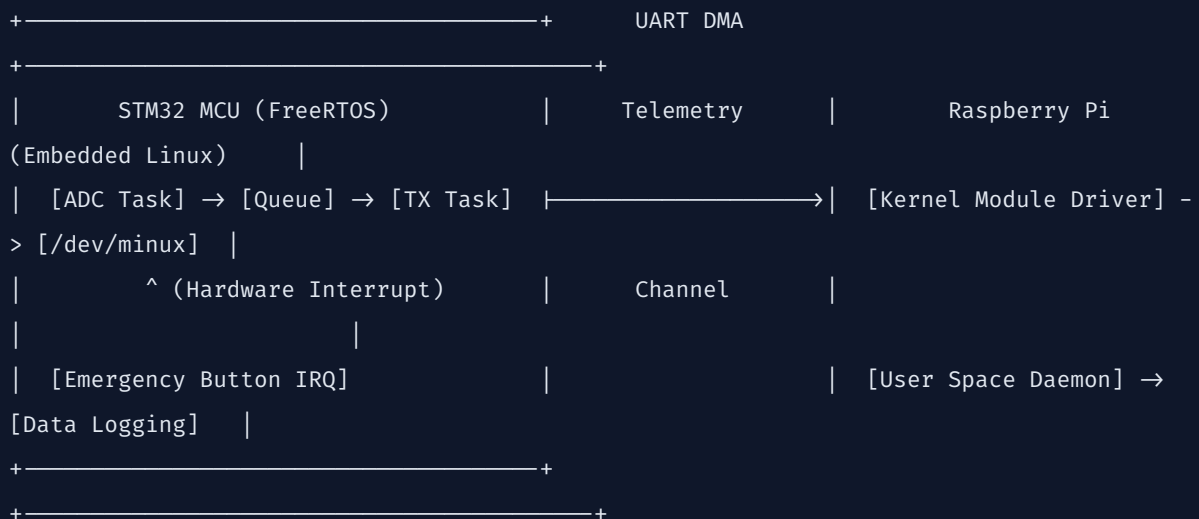
- Description: Profile database throughput under a parallel load of 100,000 operations, auditing memory leak bounds.

- Deliverable: Gprof profiling report identifying hot-spots, and Valgrind logging showing 0 bytes lost.

Project 2 (Embedded): MinuxGateway - Real-Time Industrial IoT Gateway

A real-time, closed-loop telemetry gateway utilizing FreeRTOS on an STM32 microcontroller to capture analog sensors data, communicating via DMA-UART with a custom Embedded Linux character device module running on a Raspberry Pi.

System Architecture



Core Tech Stack & Requirements

- **MCU Firmware:** FreeRTOS ports, direct register manipulation RCC/GPIO/USART, NVIC nested interrupts, DMA channel control, STM32F4.

- **Kernel Space:** Out-of-tree Linux Kernel Module (LKM), character device registration (`cdev`), file operations (`file_operations`), user-kernel space copy (`copy_to_user`), Device Tree compatible string mapping.
 - **Hardware Interfacing:** Direct UART TTL wired bridges, logic analyzers for frame debugging.
-

MNC-Style Sprint Task Breakdown (Jira Checklist)

Sprint 1: FreeRTOS Edge Firmware Setup

- **[JIRA-GW-101] MCU Register Clock & GPIO Drivers**

- Description: Initialize RCC clock dividers and configure GPIO pins for LEDs and buttons at the direct register level.
- Deliverable: `gpio_driver.c` with absolute zero HAL usage.

- **[JIRA-GW-102] FreeRTOS Multi-Tasking Core Setup**

- Description: Create two tasks: `Task_SensorRead` (priority 2) reading ADC voltage levels, and `Task_TelemetryTransmit` (priority 1) managing serial pipelines.
- Deliverable: Task synchronization loop utilizing FreeRTOS Message Queues.

- **[JIRA-GW-103] Low-Latency Emergency Interrupt (ISR)**

- Description: Configure EXTI line interrupts for an emergency stop button, invoking an ISR that immediately suspends tasks and clears power pins.
- Deliverable: Safe ISR executing in constant time with zero FreeRTOS allocations.

Sprint 2: Direct Memory Access (DMA) USART Drivers

- **[JIRA-GW-201] USART register-Level Driver**

- Description: Program USART baudrate calculation math (BRR) and control registers to configure serial frames format (8N1).
- Deliverable: `usart.c` supporting byte-level read/write.

- **[JIRA-GW-202] DMA Telemetry Channel Configuration**

- Description: Configure DMA stream channels to automatically transfer sensor data queues from SRAM to the USART transmit register, bypassing CPU overhead.
- Deliverable: Verified high-frequency telemetry output over DMA without CPU polling halts.

Sprint 3: Linux Kernel space character Driver

- **[JIRA-GW-301] Out-of-Tree Kernel Module Template**

- Description: Write a kernel module build configuration Makefile (`Kbuild`) and initialization functions (`init_module` / `cleanup_module`).
- Deliverable: Compiled loadable module (`minux_gateway.ko`) flashing cleanly.

- **[JIRA-GW-302] Character Device Registration & Dev Nodes**

- Description: Register major/minor device numbers and create character device nodes under `/dev/minux` automatically using class mappings.
- Deliverable: `/dev/minux` file entry dynamically generated upon loading module.

- **[JIRA-GW-303] File Operations & User-Kernel Copy**

- Description: Implement `file_operations` handlers, mapping unbuffered `read` requests to copy stored UART telemetry buffers to user space.
- Deliverable: `copy_to_user` memory mapping execution with proper error checking.

Sprint 4: End-to-End Integration & Boot Optimization

- **[JIRA-GW-401] User Space Logging Daemon**

- Description: Write a C-based Linux daemon that reads telemetry packets from `/dev/minux` and commits them to persistent storage.
- Deliverable: Logging daemon running asynchronously as a systemd service.

- **[JIRA-GW-402] Hardware-in-the-Loop Boot Audit**

- Description: Connect a logic analyzer, trace start sequences, and optimize kernel loading, verifying that the target starts telemetry within 8 seconds.
- Deliverable: High-resolution timing log confirming sub-8 second edge boot times.

Project 3 (Automation): MinuxHIL - Enterprise HIL Automation Framework

A fully typed, modular, and concurrent **Hardware-in-the-Loop (HIL) Test Automation Framework** written in pytest, incorporating PyVISA SCPI drivers factories, relay fault injections, binary ELF parsing, and automated Jenkins pipelines.

System Architecture

```
graph TD
    TestRunner["pytest Engine (conftest.py)"] -->|Dynamic Selection| DriverFactory["Device Driver Factory"]
    DriverFactory -->|VISA/SCPI connection| PowerSupply["Singleton Power Supply Wrapper"]
    DriverFactory -->|Serial USB interface| RelayBoard["Relay Controller (Fault Injection)"]
    TestRunner -->|Queue Telemetry| LogThread["Asynchronous Logging Thread"]
    TestRunner -->|Post-Test Compile| PDFReport["Weasyprint HTML-to-PDF Engine"]
    TestRunner -->|Archiving| CI["Jenkins / GitHub Actions pipeline"]
```

Core Tech Stack & Requirements

- **Framework:** pytest, parameterized testing matrices, conftest.py fixtures injection.
- **Interfacing:** PyVISA SCPI protocols, USB Relay serial, PySerial byte parsing.
- **Type Safety:** 100% strict type hints (`mypy --strict` compliance).
- **Firmware Audits:** ELF section sizes and symbol table analysis (`pyelftools`).
- **CI/CD:** Jenkinsfile / GitHub Actions pipeline, JUnit XML parsed results summaries, Slack notification webhooks.

MNC-Style Sprint Task Breakdown (Jira Checklist)

Sprint 1: Connection Abstractions & VISA DOMs

- **[JIRA-HIL-101] Singleton VISA Connection Manager**
 - Description: Build a thread-safe connection manager class ensuring only a single VISA resource manager session operates per instrument port.
 - Deliverable: `visa_manager.py` with 0 duplicate connection sessions.
- **[JIRA-HIL-102] Device Object Model (DOM) SCPI wrappers**
 - Description: Wrap DC Power Supplies and Digital Multimeters. Implement high-level methods (e.g. `set_voltage()`, `measure_current()`), checking inputs defensively.
 - Deliverable: `power_supply.py` and `dmm.py` drivers with strict typing.

Sprint 2: Pytest Fixtures & Fault Injection

- **[JIRA-HIL-201] Scoped Setup/Teardown Fixtures**
 - Description: Write a session-scoped `instrument_session` fixture that initiates VISA ports, and a function-scoped `power_rail` fixture that resets rails.

- Deliverable: `conftest.py` with robust, clean teardown de-energize protections.
- **[JIRA-HIL-202] USB Relay Fault Injector Driver**
 - Description: Write a serial driver for a multi-channel relay board, enabling software-driven fault injections (e.g. short-circuiting sensor buses).
 - Deliverable: `relay_driver.py` with channel toggle controls.
- **[JIRA-HIL-203] Parameterized Voltage Sweeps Tests**
 - Description: Write a pytest scenario executing a 10-step voltage regulation sweep, verifying target accuracy within 2% using parameterized matrices.
 - Deliverable: `test_regulation.py` running sweeps automatically.

Sprint 3: Advanced ELF Parsing & Asynchronous Logging

- **[JIRA-HIL-301] Pre-Flashing ELF Static memory Auditor**
 - Description: Build a validation utility using `pyelftools` that parses a compiled ELF firmware file, checks section allocations, and blocks execution if SRAM sizes exceed 90%.
 - Deliverable: `elf_auditor.py` integrated into pytest pre-flight checks.
- **[JIRA-HIL-302] Multi-Threaded Asynchronous Telemetry Logger**
 - Description: Spawn a background logging thread that consumes telemetry byte streams from PySerial, parses frames using the `struct` module, and commits them to a thread-safe queue.
 - Deliverable: `async_logger.py` with 0 frame losses.

Sprint 4: Reporting & CI/CD Pipelines

- **[JIRA-HIL-401] Click CLI Metrics compiler & PDF Generator**
 - Description: Build a CLI using `click` that parses the output CSV metrics, graphs voltage sweeps using Matplotlib, builds a styled HTML template, and compiles it via Weasyprint to a PDF report.
 - Deliverable: `report_generator.py` producing professional reports.
- **[JIRA-HIL-402] Continuous Integration Pipeline & Slack Webhook**
 - Description: Write a GitHub Actions workflow that runs your pytest suites, compiles JUnit XML reports, and sends structured build alerts to a Slack channel webhook.
 - Deliverable: `.github/workflows/hil.yml` passing lint, typing, test, and reporting stages.